

Dynamic Dispatch Simulator — Complete Project

Documentation

Medical Delivery Optimization

1. Project Overview

The Dynamic Dispatch Simulator is a proof-of-concept Streamlit web application that demonstrates automated vehicle routing for a medical delivery company operating in Paris and its suburbs. It implements a custom, hand-written greedy heuristic for the Vehicle Routing Problem with Time Windows (VRPTW) — without using any black-box optimization libraries.

Core design principle: Reliability over Efficiency. A missed hospital delivery costs more than a week of fuel savings.

2. Business Context

- **Company type:** Medium-sized medical delivery company
- **Location:** Paris and suburbs
- **Daily volume:** 40–60 deliveries, peaking on Mondays (up to 70)
- **Fleet:** 5 drivers, 6 vehicles (1 refrigerated, 2 large, 3 small)
- **Current system:** Manual Excel planning + WhatsApp coordination
- **Main problem:** Fragile planning leads to cascade failures on disruptions

Client pain points:

- Manual scheduling cannot handle real-time disruptions
 - Hospital deliveries have zero tolerance for lateness
 - Temperature-sensitive goods require refrigerated transport — no exceptions
 - Pharmacy deliveries are flexible but consume most of the daily volume
-

3. Problem Formalization

The problem is modelled as a Heterogeneous Fleet VRPTW:

- **Decision:** Assign each delivery order to exactly one vehicle in a feasible sequence
- **Objective:** Maximize the number of assigned deliveries, prioritizing critical types
- **Hard constraints:** Hospital 9am deadline, refrigerated van for cold goods, driver 10h limit, 24-hour medical transit rule
- **Soft constraints:** Time windows for Pharmacy and Temperature-Sensitive deliveries

Delivery priority order:

Dynamic Dispatch Simulator — Complete Project Documentation

Medical Delivery Optimization

1. Temperature-Sensitive → Refrigerated Van ONLY
 2. Hospital → Large Vans first
 3. Pharmacy → Small Vans first, overflow to Large Vans
-

4. System Architecture

Single file: app.py

- Section 0: Paris Travel-Time Matrix & District Data
- Section 1: Data Structures (Delivery, Vehicle)
- Section 1B: Constraint Validators
- Section 2: Fleet Creation & Delivery Generation
- Section 3: Greedy Assignment Algorithm
- Section 4: Explainability Engine + Streamlit UI
- Phase 3: CSV/Excel Data Ingestion, KPI History Tracking, Benchmark Runner
- Phase 4: Explainability Panel UI, Last-Minute Order Insertion, Route Re-Optimization

Tech stack: Python 3.9+, Streamlit, Pandas, no external solvers

5. Phase 1 — Core Constraint Enforcement

Status: COMPLETE | Date: 2026-06-05

Phase 1 transformed the original capacity-packing demo into a real VRPTW solver.

Time windows added:

- Hospital: 06:00–09:00 (HARD — never violated)
- Temperature-Sensitive: 06:00–18:00 (Soft)
- Pharmacy: 06:00–18:00 (Soft)

Driver working hours: Max 10h per driver (EU legal limit). Algorithm refuses assignment if it would exceed the limit.

24-hour medical transit rule: Each delivery records when it was loaded. Must complete within 24 hours.

Constraint validators added:

- `is_delivery_within_time_window()`
- `is_delivery_within_24h_rule()`

Dynamic Dispatch Simulator — Complete Project

Documentation

Medical Delivery Optimization

- `is_driver_within_hours()`
- `check_all_constraints()`

Test results:

- 30 deliveries → 100% assigned
 - 60 deliveries → 100% assigned
 - 70 deliveries → 82.9% assigned, 12 deferred (all Pharmacy, driver hours limit)
-

6. Phase 2 — Travel-Time Matrix & Explainability Engine

Status: COMPLETE | **Date:** 2026-06-05

Districts modeled:

- Warehouse (Le Marais), North_Paris, East_Paris, Southeast, Southwest, West_Paris

Traffic buckets:

- Morning Rush (6–10am): slowest — Warehouse to North Paris = 32 min
- Mid-Morning (10am–12pm): normal — 20 min
- Afternoon (12–5pm): fastest — 16 min

Route sequencing: `sequence_route_for_vehicle()` uses nearest-district heuristic with the travel-time matrix. Includes loading time + travel + service + return to warehouse.

Explainability engine: `generate_explainability_report()` returns the primary blocking constraint, which vans were checked and why each failed, the closest van, and what would unblock the delivery.

Repair heuristic: `repair_assign_from_van()` reassigns deliveries from a disrupted van to remaining vehicles using priority rules.

7. Phase 3 — Data Ingestion, KPI Dashboard & Benchmark Runner

Status: COMPLETE

CSV/Excel Data Ingestion:

- Required columns: id, type, boxes, district
- Optional: service_time, early_time, late_time, is_hard_window
- Type aliases accepted: hosp, pharm, temp, cold, etc.
- Auto-correction with warnings for invalid values
- Sample CSV template available for download

Dynamic Dispatch Simulator — Complete Project Documentation

Medical Delivery Optimization

KPI History Dashboard — tracks every run automatically:

- Hit rate %, total assigned/deferred, per-type counts
- Average fleet utilization %, average driver hours
- Chaos mode used per run
- Line chart (hit rate trend), bar chart (assigned vs deferred), full run log table

Benchmark Runner — 7 standard scenarios:

- Normal Monday (50 orders)
- Heavy Monday (70 orders)
- Monday + Sick Driver (50 and 70 orders)
- Monday + Fridge Breakdown (50 and 70 orders)
- Worst Case — Both Chaos (70 orders)

Runs each scenario across multiple random seeds and outputs avg hit rate, deferrals by type, and driver hours. Results downloadable as CSV.

8. Phase 4 — Polish & Production Features

Status: COMPLETE

Explainability Panel UI:

- Summary metrics: total deferred, critical deferred, pharmacy deferred
- Red alert if any Hospital or Temp-Sensitive orders were deferred
- Grouped by constraint category
- Per delivery: root cause, closest van, what would unblock it

Last-Minute Order Insertion:

- Form fields: Delivery Type, Number of Boxes, District, Service Time
- Follows same priority rules as main algorithm
- Checks capacity and driver hours before inserting
- Reports which van accepted it, or explains rejection

Route Re-Optimization Post-Disruption:

- Removes all deliveries from the disrupted van
- Finds the best remaining van by minimizing extra travel time (not just first-fit)
- Checks driver hours before inserting

Dynamic Dispatch Simulator — Complete Project Documentation

Medical Delivery Optimization

- Re-sequences all routes using the travel-time matrix
 - Reports which deliveries moved where and updated route time estimates
-

9. Algorithm Design

Greedy Priority Heuristic:

For each delivery type in order (Temp-Sensitive → Hospital → Pharmacy): For each delivery of that type: For each candidate vehicle in priority order: Estimate arrival time using travel-time matrix Check all constraints (capacity, driver hours, time window, 24h rule) If all pass: assign and update driver hours If not assigned: defer with specific constraint reason

Why greedy and not optimal? The algorithm is explainable to a non-technical CEO, easy to modify, and fast. The client's priority is reliability, not mathematical optimality.

10. Data Model

Delivery fields: id, type, boxes, time_window, early_time, late_time, is_hard_window, load_time, district, service_time

Vehicle fields: id, type, capacity, current_load, assigned_deliveries, driver_start_time, driver_hours_used, driver_hours_limit

11. Fleet Configuration

- Refrigerated-Van-1: Refrigerated, 45 boxes — Temperature-Sensitive ONLY
- Large-Van-1: Ambient, 45 boxes — Hospital deliveries
- Large-Van-2: Ambient, 45 boxes — Hospital deliveries
- Small-Van-1: Ambient, 30 boxes — Pharmacy deliveries
- Small-Van-2: Ambient, 30 boxes — Pharmacy deliveries
- Small-Van-3: Ambient, 30 boxes — Pharmacy deliveries

Chaos mode: Sick Driver → Small-Van-3 capacity = 0. Fridge Breakdown → Refrigerated-Van-1 capacity drops to 22 boxes.

12. Constraint System

- Vehicle capacity — Hard — Phase 1
- Hospital 9am deadline — Hard — Phase 1
- Driver 10h limit (EU law) — Hard — Phase 1

Dynamic Dispatch Simulator — Complete Project Documentation

Medical Delivery Optimization

- 24-hour medical transit rule — Hard — Phase 1
 - Refrigerated van for cold goods — Hard — Phase 1
 - Soft time windows for Pharmacy and Temp-Sensitive — Soft — Phase 1
 - Paris traffic-based travel times — Estimation — Phase 2
-

13. Key Design Decisions & Trade-offs

- **Reliability over efficiency:** Client stated a missed hospital delivery costs more than a week of fuel savings
 - **Custom greedy over OR-Tools:** Explainable to non-technical CEO, easy to modify
 - **Fixed priority order over dynamic weighting:** Simple, predictable, defensible
 - **Defer pharmacy over split routes:** Pharmacy orders are same-day flexible
 - **Single file over multi-module:** Easier for student team to manage and present
 - **Static matrix over Google Maps API:** No API cost, realistic enough for proof-of-concept
-

14. Demo Scenarios

- Normal Monday (50 orders, no chaos) → ~100% hit rate
 - Heavy Monday (70 orders, no chaos) → 85–100%, some pharmacy deferred
 - Sick Driver (50 orders) → 90–100%, hospitals always protected
 - Fridge Breakdown (50 orders) → 90–100%, temp-sensitive may defer if overflow
 - Worst Case (70 orders, both chaos) → 50–70%, only pharmacy ever deferred
-

15. Installation & Running

```
cd "c:\Users\...\RLA"
```

```
pip install -r requirements.txt
```

```
python -m streamlit run app.py
```

Open browser at: <http://localhost:8501>

Requirements: streamlit, pandas, openpyxl

16. Production Roadmap

- Real data integration (High) — ingest orders from email/API

Dynamic Dispatch Simulator — Complete Project Documentation

Medical Delivery Optimization

- Google Maps travel times (High) — replace static matrix
 - Improved solver (Medium) — simulated annealing or MILP
 - Real-time driver tracking (Medium) — live disruption handling
 - Driver mobile app (Low) — manifests and ETA updates
 - Monitoring and KPI alerts (Medium) — automatic alerts on missed windows
-

Team: Kevin Otah Ogbusuo · Mishvaba Hitensinh Vaghela · Pritheesh Selvaraja Kumar